

Temel Kavramlar ve AI Mimarisi

Bu bölümde üç ayrı konuya baktım: Dar AI ile AGI arasındaki fark, embedding/vektör veritabanı mantığı ve eğitim (training) ile çıkarımın (inference) birbirinden ne kadar farklı olduğu.

Dar Yapay Zekâ (ANI) ile Yapay Genel Zekâ (AGI)

Bugün kullandığımız bütün büyük dil modelleri (ChatGPT, Claude falan) hâlâ Dar AI (ANI) sınıfındaymış — ilk başta bana çok "genel" gibi görünüyordu ama meğer hepsi tek bir şeyde, "bir sonraki kelimeyi tahmin etmede", uzmanlaşmış; sadece bu görev o kadar geniş bir yelpazede işe yarıyor ki genel zekâ gibi hissettiriyor. **AGI** dediğimiz şey ise insan gibi, önceden hiç görmediği bir görevi de öğrenip genelleayebilen varsayımsal bir sistem.

AGI'ye ulaşmak için önümüzde duran engelleri araştırınca şunları buldum: gerçek nedensellik kurabilme (bugünkü modeller daha çok korelasyon öğreniyor, "neden-sonuç" değil), sürekli öğrenme (yeni bir şey öğrenirken eskisini unutmama problemi — buna *catastrophic forgetting* deniyor), örüntü tanıma ile mantıksal akıl yürütmeyi güvenilir şekilde birleştirebilme, ve insana kıyasla çok daha az veri/enerjiyle öğrenebilme.

Vektör Temsilleri (Embeddings) ve Vektör Veritabanları

Embedding kavramını araştırırken en basit anlatımı şuydu: bir metnin anlamını sabit uzunlukta bir sayı dizisine (vektöre) sıkıştırıyorsun. Anlamca yakın iki metin bu vektör uzayında da birbirine yakın duruyor — mesela "köpek" ile "kedi" vektörleri, "köpek" ile "bilgisayar"a göre birbirine çok daha yakın çıkıyor.

Normal bir veritabanı (SQL gibi) tam eşleşmeyle arama yapıyor (`isim='Ahmet'` gibi). Vektör veritabanı ise "en yakın komşu" mantığıyla çalışıyor: sorduğun şeye *anlamca* en yakın kayıtları buluyor. Pinecone, ChromaDB, Qdrant gibi araçları karşılaştırdım — hepsi de milyonlarca vektör arasında tek tek karşılaştırma yapmak yerine (çok yavaş olurdu) yaklaşık bir arama algoritması (ANN, genelde HNSW) kullanıyor.

KISACA KARŞILAŞTIRDIĞIM ÜÇ ARAÇ		
ARAÇ	KARAKTERİ	ÖNE ÇIKTIĞI YER
Pinecone	Tamamen yönetilen, bulut-native	Kurulumuz ölçeklenebilirlik
ChromaDB	Açık kaynak, hafif	Lokal geliştirme / prototipleme
Qdrant	Açık kaynak + yönetilen seçenek, Rust tabanlı	Güçlü metadata filtreleme

Model Eğitimi (Training) ile Çıkarım (Inference)

Bu ikisini ilk duyduğumda aynı şey sanmıştım ama hiç değiller. **Training**, modelin parametrelerinin veriyle güncellendiği, hem ileri hem geri yayılım gerektiren, günlerce-haftalarca sürebilen, bir sürü güçlü GPU isteyen bir süreç. **Inference** ise artık eğitimi bitmiş, donmuş bir modelin sadece tek yönlü (ileri) bir hesap yaparak cevap üretmesi — milisaniyeler sürüyor ama her kullanıcı isteğinde yeniden çalıştığı için, ölçek büyüdükçe toplamda training'den bile daha pahalıya gelebiliyormuş; bu benim için sürpriz oldu.

TRAINING VS. INFERENCE		
BOYUT	TRAINING	INFERENCE
Sıklık	Bir kez / periyodik	Her kullanıcı isteğinde
Süre	Saatler – haftalar	Milisaniyeler – saniyeler
Donanım	Çoklu GPU/TPU, yüksek VRAM	Tek GPU, hatta CPU (küçük modelde)
Maliyet yapısı	Yüksek, tek seferlik sabit maliyet	Düşük ama ölçekle katlanan değişken maliyet